

MIPSolvers 原生 IPM 体系增强路线图

——2023–2026 前沿算法调研与并行化失效诊断

基于 docs/native_ipm_audit_report.pdf 审计结论的续篇

2026 年 8 月 2 日

摘要

本报告解决一个具体问题：为什么当前并行计算不能为该 IPM 体系加速，以及如何用 2023–2026 年的最新算法成果修复它。诊断结论：OpenMP 仅包裹了装配级 $O(\text{nnz})$ 循环，而占每迭代 70–90% 成本的稀疏分解全部串行（EigenSparseLU/KLU/UMFPACK 顺序执行、MUMPS 走 mpiseq 顺序兼容层、CHOLMOD 未配置并行）——按 Amdahl 定律，现状并行收益上限仅约 1.1–1.4 $\times$ 。增强路线的核心判断：本代码库与当前 GPU-IPM 前沿（HyKKT/LiftedKKT 凝聚 KKT + cuDSS）的接口距离比表面看起来近得多——凝聚形式 $W = H + J_h^\top (M/S) J_h$ 、SPD 认证、CHOLMOD 封装、稠密 Schur 求解、“analyze-once/factorize-many”缓存契约均已存在；真正的缺口是（i）分解后端串行且不定、（ii）无真实惯性、（iii）审计发现的两个串行热点（线搜索全 Jacobian 重评、Merit 路径每迭代 ≤ 4 次重复分解）封死了任何并行化的收益上限。

目录

1	并行化失效诊断：为什么现在并行不加速	1
1.1	当前并行触点全清单	1
1.2	串行瓶颈：分解后端全部顺序执行	1
1.3	Amdahl 定量估计	2
2	前沿调研（2023–2026）：与本体系相关的最新 IPM 算法	2
2.1	GPU 凝聚 KKT 内点法：HyKKT / LiftedKKT / MadNLP	2
2.2	cuDSS：GPU 稀疏直接求解器	3
2.3	CPU 侧：DAG 任务并行的对称不定分解	3
2.4	一阶 GPU LP 方法：PDLP $\rightarrow$ cuPDLP $\rightarrow$ cuPDLP+	3
2.5	批量与结构化并行	3
3	前沿 $\rightarrow$ 本代码库映射	4
4	分层增强路线图	4
4.1	T1：CPU 快速收益（1–2 周，低风险）	4
4.2	T2：结构级（1–2 月，中风险）	5
4.3	T3：GPU 化（1 季度以上，需 NVIDIA GPU + CUDA 工具链）	5

1 并行化失效诊断：为什么现在并行不加速2

5 预期收益与风险（诚实估计）6

6 验证计划6

1 并行化失效诊断：为什么现在并行不加速

1.1 当前并行触点全清单

全库 OpenMP 在 IPM 内核中的使用点(编译开关 MIPSOLVERS_USE_OPENMP 默认 ON,CMakeLists.txt:439

位置	并行内容	量级
cones.cpp:324	NT 缩放的逐锥块计算	$O(\sum q + \sum p^3)$, 小
conic_ipm_solver.cpp:521,553,724	Schur ($G^T H^{-1} G$) 装配	$O(\text{nnz})$ 级
ipm_lp_solver.cpp:1761 等	Θ 构造、残差合并等逐分量循环	$O(n)$

线程数控制：锥求解器 num_threads 默认 0（库默认），OmpThreadsGuard 仅在 > 0 时设定（证据：conic_ipm_solver.hpp:31, conic_ipm_solver.cpp:65--81）。

1.2 串行瓶颈：分解后端全部顺序执行

每迭代主导成本是 KKT/正规方程分解，而所有默认后端均为**串行**：

后端	类型	并行性	位置
EigenSparseLU	非对称 LU	串行	linear_solver.cpp:2
KLU/UMFPACK/SuperLU	非对称 LU	串行	linear_solver.cpp:6
MKL PardisoLU	非对称 LU	仅 MKL 内部线程（且为非对称路径）	linear_solver.cpp:1
MUMPS	对称不定 LDL^T	mpiseq 顺序层, MPI_Init 为空操作	linear_solver.cpp:2
CHOLMOD（锥 KKT）	SPD LL^T/LDL^T	未配置 GPU/多线程，仅超节点 GEMM 借 BLAS 线程	cholmod_ldlt.cpp:37

注意 MUMPS 即便编译了并行,当前也通过 mpiseq 以单进程方式驱动(证据:linear_solver.cpp:248--257 注释)。于是**分解这一主导环节零并行**。

1.3 Amdahl 定量估计

IPM 每迭代成本近似 $T = T_{fact} + T_{asm} + T_{eval} + T_{ls}$ 。对中大型稀疏问题，直接法分解 T_{fact} 通常占 70–90% (Lp 正规方程 $O(\text{nnz}(N)^{1.5})$ 级, NLP 增广 KKT 更高)。当前并行只作用于 T_{asm} 的一部分（且装配本身已有高效的 scatter-map 缓存，占比进一步压低），故

$$S_{max} \leq \frac{1}{(1 - p)} \approx 1.1\text{--}1.4 \times \quad (p \leq 0.3 \text{ 的可并行份额, 且其中仅一部分被实际并行}), \quad (1)$$

与“并行无加速”的观察一致。此外审计发现两个**串行热点**会进一步封顶任何并行收益，必须先修：

- **H1**: NLP-IPM 线搜索每次回退重算全部 Jacobian (ipm_solver.cpp:849--884 → 736–753)——Ipopt 只评 f, g, h ;
- **H2**: Merit 路径 reduced-KKT 每迭代 ≤ 4 次对同一 W 全量重新分解(ipm_solver.cpp:2317,2357,2408 → kkt_system.cpp:283--354)。

2 前沿调研 (2023–2026): 与本体系相关的最新 IPM 算法

2.1 GPU 凝聚 KKT 内点法: HyKKT / LiftedKKT / MadNLP

Pacaud、Shin、Montoison、Schanen、Anitescu 的系列工作是当前 GPU-NLP-IPM 的主线¹:

- **核心观察**: KKT 直接分解的数值主元 (pivoting) 本质串行, 是 GPU 化的根本障碍; 将不定 KKT 凝聚为 SPD 系统后可用无主元 Cholesky, 天然适合 GPU。
- **HyKKT**: 将 $(1, 1)$ 块 ($H_\gamma = H +$ 正则, SPD) 用 GPU Cholesky 分解, 等式 Schur 补 $S_\gamma = J_g H_\gamma^{-1} J_g^\top$ 用 **共轭梯度 (CG)** 迭代求解——完全无主元; 分析表明凝聚系统的病态是“结构化的”, 精度损失可控。
- **LiftedKKT**: 不等式提升为等式 + 松弛后凝聚为单一 SPD 系统, 直接 Cholesky。
- 实现在 MadNLP.jl (filter 线搜索 IPM, 与本体系同框架) + ExaModels.jl (GPU 导数评估); 经典蒸馏塔算例**每迭代提速 25 倍**²; 已用于 67.4 万维电网问题。

2.2 cuDSS: GPU 稀疏直接求解器

NVIDIA cuDSS (2024) 提供 GPU 驻留的稀疏直接分解; 其契约是**分析阶段 (METIS 排序, CPU) 一次、数值分解在 GPU 反复重用**³——与本代码库 SparseKKTCache 的“结构指纹缓存 + analyze once / factorize many”契约 (证据: kkt_system.cpp:74--148, 206--242) **完全同构**, 移植阻抗极低。

2.3 CPU 侧: DAG 任务并行的对称不定分解

不引入 GPU 的现实路线。HSL MA86 (OpenMP、DAG 任务调度、对称不定)⁴、HSL MA87/MA97 (DAG 型, MA86 的不定对应物)、SPRAL/SSIDS (CPU+GPU 混合, 且**能提供惯性**——HSL 邮件列表明确指出 Ipopt 场景下只有 SPRAL 类能提供所需惯性信息⁵)。Ipopt 官方支持 ma86/ma97/spral 后端⁶。GALAHAD SLS 提供统一接口封装 MA57/MA86/MA97/SSIDS/MUMPS/PARDISO/PaStiX⁷——这正是本库 SparseLinearSolver 接口应该长成样子。

2.4 一阶 GPU LP 方法: PDLP $\rightarrow$ cuPDLP $\rightarrow$ cuPDLP+

对超大 LP (特别是 MIPLIB 松弛), 无分解的一阶方法已成第三极: cuPDLP (重启 PDHG, 仅稀疏 matvec, GPU 全并行)⁸; cuPDLP+ 引入 Halpern 重启 + 反射、新重启判据、PID 原

¹Pacaud et al., *Condensed-space methods for nonlinear programming on GPUs*, arXiv:2405.14236 (2024)。

²Shin et al., *GPU-accelerated dynamic nonlinear optimization with ExaModels and MadNLP*, arXiv:2403.15913 (2024)。

³A. Montoison, CUDSS.jl 讨论 (2024); NVIDIA cuDSS 文档。

⁴HSL MA86 规范: *Sparse symmetric indefinite system using OpenMP*。

⁵COIN-OR Ipopt 列表: *Sparse Linear Solver on GPU for IPOPT* (J. Hogg)。

⁶Ipopt 官方选项文档 (linear_solver 一节)。

⁷GALAHAD SLS 文档。

⁸Lu & Yang, *An Overview of GPU-based First-Order Methods for LP*, arXiv:2506.02174 (2025)。

始权重更新，在 MIPLIB 松弛上再快 $2-4\times^9$ 。局限同样明确：精度有限（ 10^{-4} 级实用）、无基解、暖启动能力弱——在 B&C 内部不能替代 IPM/单纯形，但可作为超大独立 LP 的前置粗解器。本库已有 NativePDLAdapter (src/engine/solver/native/lp/pdip_solver.cpp)，升级路径现成。

2.5 批量与结构化并行

- **Many-problems-one-GPU**: 将大量中小 NLP 批量驻留单 GPU 并行求解¹⁰——直接对应 B&C 强分支/泵浦中成批节点 LP 的场景（本库 solve_cached_bound_change_batch 当前为串行循环，ipm_lp_solver.cpp:3392）。
- **块三对角 Schur 分解**（动态/NMPC 结构）：自动检测块结构 + GPU batched LAPACK，性能对比 MA57/CHOLMOD/cuDSS 有优势¹¹——对应 SCUC/OPF 的时段耦合结构（本库 SCUC 元数据完备）。
- **PIPS-IPM/PIPS-NLP**: 块角结构问题的 Schur 补分解 + MPI 并行，是结构化大规模 IPM 的经典范式（SCUC 多场景/多时段可直接套用）。
- **QOCO-GPU**: GPU 加速的二次目标 SOCP 内点法¹²——证明锥 IPM（本库最强内核）GPU 化路线同样成立；其 KKT $K = G^T H^{-1} G + \delta I$ 为 SPD，与 cuDSS/CHOLMOD-GPU 直接兼容。

3 前沿 → 本代码库映射

前沿	本库已有基础（锚点）	缺口
HyKKT 凝聚 +CG	凝聚 $W = H + J_h^T (M/S) J_h$ 装配 (ipm_solver.cpp:1260--1267); SPD 认证 (kkt_system.cpp:384--407); 小 Schur 稠密求解 (kkt_system.cpp:283--354)	无 SPD 后端复用（每次新建分解器）；无 CG/Krylov 求解器；Schur 仅用于 $m_{eq} \leq 8$
cuDSS GPU 分解	analyze-once 缓存契约 (kkt_system.cpp:206--242)；锥 KKT 为 SPD (CHOLMOD 路径)	无 GPU 后端；值数组布局需适配 cuDSS 接口
MA86/97、SPRAL 并行不定分解	后端抽象 SparseLinearSolver (linear_solver.hpp:12--19)	默认链全为非对称串行 LU；MUMPS 被 mpiseq 串行化
真实惯性驱动 δ_W	MUMPS INFOG(12) 已封装 negative_eigenvalues() (linear_solver.cpp:397--401)	MUMPS 非默认；现有“惯性”是 $Z^T W Z$ 稠密特征解 (nullity ≤ 128) 或全空间凸化
cuPDL+ 一阶 LP	NativePDLAdapter 已存在 (pdip_solver.cpp)	无 Halpern 重启/PID 权重/GPU matvec

⁹Lu, Peng & Yang, *cuPDL+*, arXiv:2507.14051 (2025)。

¹⁰Scaling Nonlinear Optimization: Many Problems One GPU, arXiv:2606.26341 (2026)。

¹¹AICHe 2025: GPU-Accelerated Solution of Block-Tridiagonal Systems for Large-Scale Optimization。

¹²QOCO-GPU: A Quadratic Objective Conic Optimizer with GPU Acceleration, arXiv:2603.29197 (2026)。

前沿	本库已有基础（锚点）	缺口
批量节点 LP	批量接口存在但串行 (<code>ipm_lp_solver.cpp:3392--3497</code>)	无批量分解/batched BLAS
SCUC 块结构	UCGenHint 元数据完备 (<code>problem_types.hpp:281--406</code>)	IPM 未利用块角/块三对角结构

4 分层增强路线图

4.1 T1: CPU 快速收益（1–2 周，低风险）

1. **T1.1 修复串行热点（前置，否则一切并行收益被封顶）**: H1——线搜索试点只评 f, g, h (Jacobian 每迭代一次，试算 θ, φ 只需函数值); H2——Merit 路径跨 RHS 复用同一 W 的分解 (predictor/corrector/Gondzio 共用一次分解)。预期 $1.2\text{--}2\times$ (串行)。
2. **T1.2 接入并行对称不定后端**: 将 `make_default_sparse_solver` 的 NLP-KKT 默认改为(按可得性) MKL PARDISO 对称不定路径 (LDL^T , 多线程)、多线程 MUMPS (脱离 mpiseq, 配多线程 BLAS; OpenMP 版 MUMPS)、或 SSIDS/SPRAL。预期分解环节 $2\text{--}6\times$ (8–16 核)。
3. **T1.3 真实惯性驱动 δ_W** : 以 MUMPS/SSIDS 的负主元计数 (`negative_eigenvalues()` 已封装) 替换 $Z^T W Z$ 稠密特征解路径, W 循环回到 Wächter–Biegler 标准形式; 同时消除 nullity > 128 时的全空间凸化副作用。
4. **T1.4 稠密路径换多线程 BLAS/LAPACK**: LCQP 与 LP-IPM 的稠密/带状 Cholesky (自研 `banded_chol_*`) 换 LAPACK `dpbsv/dpotrf` (Accelerate/MKL 均已多线程); 稠密正规方程 AA^T 用 `dsyrk`。

4.2 T2: 结构级（1–2 月，中风险）

1. **T2.1 HyKKT 化（无主元化，为 GPU 铺路）**: 在现有凝聚路径上，将 $W + \delta_W I$ (已有 SPD 认证逻辑) 用 CholmodLDLT 一次分解、多 RHS 复用; 等式 Schur 补 $J_g(W + \delta I)^{-1} J_g^T + \delta_C I$ 从 “ $m_{eq} \leq 8$ 稠密” 推广为任意 m_{eq} 的 CG (matvec = 两次三角回代 + J_g 稀疏乘, 全部 GPU 友好)。收益: KKT 全流程无主元 $\Rightarrow$ 可直接落 cuDSS; 并消除非对称 LU 的 $2\times$ 开销。这是性价比最高的一步。
2. **T2.2 SCUC 块结构 Schur 分解**: 利用 UCGenHint 的时段结构，将 KKT 重排为块角/块三对角形，对角块独立分解 (OpenMP 任务并行) + 耦合 Schur 串行小系统 (PIPS-IPM 范式; 动态结构可参考 AICHE-2025 块三对角 GPU 方案)。
3. **T2.3 B&C 批量节点 LP**: `solve_cached_bound_change_batch` 改为多线程分派 (不同 bound-change 独立求解, 线程安全依赖每线程 `CachedState`); 强分支候选 LP 同理批量化。

4.3 T3: GPU 化（1 季度以上，需 NVIDIA GPU + CUDA 工具链）

1. **T3.1 cuDSS 后端**: 实现 `SparseLinearSolver` 的 cuDSS 版本（CPU/METIS 分析一次 → GPU 反复分解/回代），首先服务锥 IPM 的 SPD KKT（改动最小、收益最直接）。
2. **T3.2 凝聚 SPD + 全 GPU KKT**: T2.1 的凝聚路径整体落 GPU（cuDSS Cholesky + GPU CG）；导数矩阵经现有 scatter-map 在 GPU 上填充值数组。参照 MadNLP 的全驻留设计，避免每迭代 H2D 拷贝。
3. **T3.3 NativePDLP 升级**: 引入 cuPDLP+ 的 Halpern 重启、反射、PID 原始权重；matvec 移 GPU。定位：超大独立 LP 的粗解器，不替代 B&C 内 IPM。
4. **T3.4 GPU 导数评估**: 现回调式 `NLPModel` 无法 GPU 化；长期需表达式图（参照 ExaModels 的 SIMD/SIMT 单遍评估）。此项成本最高，建议仅在明确大规模 NLP 业务需求时立项。

5 预期收益与风险（诚实估计）

路线	预期加速	主要风险	备注
T1.1 热点修复	1.2–2×（串行）	低	必须先做
T1.2 并行不定后端	分解 2–6×	中（HSL 许可/MKL 依赖）	端到端受 Amdahl 限制
T1.3 真实惯性	稳健性为主	低	附带去掉稠密特征解成本
T2.1 HyKKT 化	1.5–3× + GPU 通行证	中（Schur-CG 收敛依赖 δ_C ）	需 IR 兜底（已有 2 步 IR）
T2.2 结构分解	视块数 2–10×	高（重排/映射复杂）	仅结构化问题
T2.3 批量节点 LP	近线程数线性	中（线程安全）	B&C 侧收益
T3 GPU	大问题 5–25×/迭代	高（平台、精度、惯性缺失）	分解 > 0.1 s/次才划算（HSL 经验）

通用告诫：(i) 凝聚路径失去分解惯性， δ_W 策略需重新设计（HyKKT 论文用固定正则 +IR，本库的约化空间证书恰可补位）；(ii) GPU 仅在单次分解足够大时回本；(iii) 一阶方法精度上限决定其不能进 B&C 内环。

6 验证计划

1. **正确性门槛**: `tests/test_ipm_solver.cpp`、`tests/test_conic_ipm.cpp` 全绿；新增 KKT 残差 $\|Kz - b\|_\infty$ 与 KKT 误差回归测试（每后端）。
2. **性能基准**: netlib/MIPLIB LP 松弛（LP-IPM）、CUTEst 中选 10 个中小 + 2 个大 NLP（NLP-IPM）、SDPLIB 若干（锥）；记录每迭代分解/装配/线搜索分项耗时（代码已有 verbose 计时基建）。
3. **并行扩展性**: 1/2/4/8/16 线程扫描，报告分解环节与端到端两组加速比（现状只有后者无意义）。